Week 6: Building the Frontend: React 2

Problem Statement

This week you will continue building the frontend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

In this assignment, you will enhance the UI by implementing React components for Threads and Comments, style them with Bootstrap, and use GitHub Copilot's Agent Skills to assist with component implementation and styling.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
3. As you make changes throughout this tutorial, remember to regularly commit your work.

These are the main components of the project:

```
src
├── App.jsx          Root component of the Application
├── main.jsx         An entry point that renders the App component
├── components/      Reusable UI components (ThreadCard, CommentList,
VoteButtons, etc.)
├── pages/           Page components (Login, Register, Home, ThreadPage)
└── services/        API service modules (threadService, commentService,
userService)
```

This session is structured in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Components for Threads and Comments	No AI Tools
Part 2	AI-Assisted Development	Agent Skills for Styling and Implementing Components, Integrated Browser for UI context, Unit Tests	GitHub Copilot

In **Part 1** you will build React components manually, without AI assistance. This ensures you understand the core React concepts.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

In **Part 2** we will use GitHub Copilot for building React components and more.

By the end of this session, you will be able to:

- Build React components for a social media app
- Use GitHub Copilot's Agent Skills to assist with component implementation and styling
- Understand how to iteratively refine UI components with AI assistance
- Generate unit tests for React components using agents

Part 1: Manual Implementation of React Components

In this part, you will build the core React components for the Threads and Comments sections of the app without using any AI tools.

1.1 Project Setup

Open a terminal in the root of the repository and run the following commands:

```
npm install
```

This should install all the necessary dependencies. Start the development server to confirm everything is working:

```
npm run dev
```

Open `http://localhost:5173` in your browser.

1.2 Implementing the Components

Components like `ThreadPage.jsx`, `ThreadCard`, `CommentList`, `CommentForm`, and `VoteButtons` are already implemented. Complete the sections marked with `/* Your Code Here */` in the following two components:

1. **`ThreadList.jsx`** - Render a list of `ThreadCard` components from the threads array
2. **`Home.jsx`** - Display the `ThreadList` component with the recent threads data

Part 2: AI-Assisted Development with GitHub Copilot

In this part, you will use GitHub Copilot to assist with implementing and styling the React components you built in Part 1. You will also generate unit tests for these components using Copilot.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

2.1 Updating Custom Instructions for Frontend Development

Since we are using Bootstrap for styling, we will update our custom instructions to reflect this. Open the `AGENTS.md` (or `copilot-instructions.md`) file and modify the section on `Styling` as follows (or generate a new instructions file and add the section below):

```
#### Styling
1. React-Bootstrap components over raw HTML:
   - Card instead of div wrappers
   - Button instead of button
   - Badge instead of span with badge classes
   - Form components instead of raw inputs
   - Container / Row / Col for layout

2. Prefer Bootstrap utility classes over custom CSS:
   - Spacing: m-*, p-*, gap-*, mb-*, mt-*
   - Layout: d-flex, justify-content-*, align-items-*
   - Typography: fw-bold, text-muted, small
   - Borders: border, rounded, shadow-sm
   - Backgrounds: bg-light, bg-body-tertiary

3. Do NOT:
   - Invent unnecessary custom CSS classes
   - Use inline styles unless unavoidable
   - Add unnecessary wrapper divs
   - Over-nest layout structures

4. Maintain:
   - Responsiveness
   - Proper spacing rhythm (prefer gap-2, gap-3, mb-3)
   - Clear visual hierarchy and semantic structure
   - Accessibility (aria-label, proper button types)
```

Save the file. These updated instructions will guide Copilot to generate code that is consistent with Bootstrap styling conventions.

We can test this by asking Copilot to generate a component with styling. For example, you could ask:

```
Generate a Forgot Password component that displays a form with an email
input and a submit button.
```

Copilot should now generate a component that uses React-Bootstrap components and utility classes for styling, following the guidelines in our custom instructions.

2.2 Augmenting Agents with Skills

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

Agent Skills are pre-built capabilities that an agent can invoke when relevant to perform a specialised task. These are reusable folders of instructions, scripts and resources that transform general purpose agents into specialist agents.

Unlike custom instructions which mainly define the style and approach for code generation, Agent Skills can include specific logic, templates, and resources to perform complex tasks. Here are some use cases of Agent Skills:

- **Code Fixing and Linting**: You can create a "Code Quality" skill that includes linting rules and best practices for React development. When you ask the agent to review or fix code, it can use this skill to identify issues and suggest improvements.
- **Styling with Agent Skills**: You can create a "Bootstrap Styling" skill that includes instructions and templates for using React-Bootstrap components and utility classes. When you ask the agent to style a component, it can invoke this skill to generate code that follows Bootstrap conventions.
- **Testing React components with GenAI**: You can create a "React Testing" skill that includes templates and instructions for writing unit tests with Jest and React Testing Library. When you ask the agent to generate tests for a component, it can use this skill to produce comprehensive test cases.

Let us use a popular pre-built skill called the React Code Fix and Linter skill. This skill automates code formatting and linting checks for React code to ensure that the generated code adheres to best practices and is free of common errors before committing it to the codebase.

1. To create this skill, in the .github folder, create a new folder called `skills`. Inside the `skills` folder, create a new folder called `fix` and create a file called `SKILL.md` with the following content:

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

```
---
name: fix
description: Use when you have lint errors, formatting issues, or
before committing code to ensure it passes CI.
---

# Fix Lint and Formatting

## Instructions

1. Run `npm run prettier` to fix formatting
2. Run `npm run lint` to check for remaining lint issues
3. Report any remaining manual fixes needed

## Common Mistakes

- **Running prettier on wrong files** - `npm run prettier` only
formats changed files
- **Ignoring lint errors** - These will fail CI, fix them before
committing
```

2. This skill uses `prettier` and `eslint` to automatically fix formatting and linting issues in the code. Let us make sure these tools are set up in our project. In a fresh conversation in Agent Mode, ask Copilot:

```
Install Prettier and ESLint for this project. Add scripts to
package.json for running them on changed files.
```

   Copilot should generate the necessary code to install these tools and add the following scripts to your `package.json`:

```
"scripts": {
  "prettier": "prettier --write .",
  "lint": "eslint ."
}
```

3. Now, whenever you want to ensure your code is properly formatted and free of linting issues, you can simply ask Copilot to "Use the fix skill" before committing your code. The agent will run the necessary commands and report any remaining issues that require manual attention. Try out the skill by asking:

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

```
Use the React fix skill to ensure my code is clean before committing.
```

The agent should respond by running the Prettier and ESLint commands, fixing any issues it can, and reporting any remaining problems that need manual fixes.

2.3 Visual Context with the Integrated Browser

We have seen how to use screenshots from external websites as context to the agent. Another powerful technique is using **your own running app** as a visual reference. VS Code's built-in **Integrated Browser** lets you view a live webpage inside the editor — and you can attach screenshots of it directly to Copilot Chat.

Opening the Integrated Browser

1. Make sure the development server is running: `npm run dev`
2. Open the Integrated Browser:
    - Press `Ctrl+Shift+P` / `Cmd+Shift+P` to open the Command Palette
    - Type `Browser: Open Integrated Browser` and press `Enter`
    - Enter the URL `http://localhost:5173`
3. The Integrated Browser will appear in a new tab in the editor.

    Note: The Integrated Browser updates automatically when Vite hot-reloads after a code change. This creates a live feedback loop — you change the code, and the browser updates immediately without switching windows.

Referencing Browser Content in Copilot Prompts (Agent Mode)

1. In the Integrated Browser, click on the `Add Element to Chat` button on the top right corner, next to the URL bar.
2. Now select an element from the page — for example, click on the badge that shows the subreddit name in a thread card.
3. Notice that the HTML and CSS context of the badge along with a screenshot of that element is now attached as context in the Copilot Chat input box.
4. In **Agent Mode**, you can now ask Copilot to make changes to that element using the visual and code context from the browser. For example, you could type:

```
1. Make the corners of the badge sharp.
2. Change the badge background to a uniform dark green without a
gradient.
3. Keep the text inside the badge in lowercase
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

5. Notice how the agent uses the attached context to directly modify the CSS for that badge element, and you can see the changes reflected immediately in the Integrated Browser.

Try selecting different elements and asking for other changes to see how the agent can use the visual context to make precise modifications.

This technique is especially powerful for styling and layout issues, where describing the problem in words can be difficult. By showing the agent exactly what you see, you can get much more accurate and relevant code changes.

Key Takeaway: The Integrated Browser lets you use your **actual running app** as context — not a static wireframe. This is especially useful for layout bugs that are immediately obvious visually but hard to describe in words.

2.4 Generate Unit Tests with GenAI

Use the react-testing-agent from the previous week to generate unit tests for your React components. We had created the agent with the following instructions:

```
/create-agent Create an agent called 'react-testing-agent' that creates
unit tests for React components. Use 'React Testing Library' as the testing
library and Vitest as the test runner. Keep all the tests in a separate
top-level tests/ directory. Keep the agent workspace scoped.
```

1. Select the react-testing-agent from the Agent selection drop-down.
2. Ask the agent to generate unit tests for a specific component, for example:

```
Generate unit tests for the ThreadCard component. Cover rendering,
props, and interactions.
```

2.5 Visualizing Prompt Caching with the Cache Explorer

Every time you send a message, the harness rebuilds one long request — your system prompt, instructions, tool and skill definitions, and the full conversation history — and appends your new message to it. When a provider supports **prompt caching**, it can reuse the unchanged beginning of that request (the **prefix**) instead of reprocessing those tokens, which lowers both latency and token cost. A change *early* in the request, however, shrinks how much of the prefix is reusable and causes a **cache miss**.

VS Code ships a built-in **Cache Explorer** that compares two consecutive requests in a chat session and shows you the cache-hit percentage, the reused input tokens, and — most usefully — the exact point where the two requests first stop matching. In this task you will trigger a cache hit and a cache miss on your ThreadHive project and use the Cache Explorer to see what changed.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

> **Note:** The Agent Debug Logs panel is currently in **Preview**, and prompt caching is provider- and model-dependent. If the Summary view does not report cache information, the selected model may not expose it — switch to a model that does before continuing.

Enable the Agent Debug Logs

1. Open **Settings** (`Ctrl+,` / `Cmd+,`) and search for `agentDebugLog`.
2. Enable the experimental setting `github.copilot.chat.agentDebugLog.fileLogging.enabled`.
3. **Reload the window** (Command Palette → `Developer: Reload Window`) so the setting takes effect. Capture is not retroactive, so logging must be on *before* you send the requests you want to inspect.

Observe a cache hit

1. Start a **fresh chat session** in Agent Mode. Send a first request about one of your Part 1 components, for example:

   ```
   Explain what the ThreadCard component renders and list the props it expects.
   ```

   This is the first request in the session, so there is nothing to reuse yet — it *writes* the cache.

2. In the **same session**, send a short follow-up that keeps the earlier prefix intact:

   ```
   Now describe how VoteButtons is used inside it.
   ```

   Because the system prompt and the previous turn are unchanged, most of the prefix can be reused.

3. Open the Cache Explorer:

   - Select the ellipsis (`...`) menu in the Chat view and choose **Show Agent Debug Logs**.
   - Select the session description in the breadcrumb to open the **Summary** view.
   - Select **Cache Explorer**.

4. In the turn list, select your second request. In the main area, read the **Cache performance** section — you should see a high cache-hit percentage and a large number of reused input tokens. This is prompt caching working in your favour.

Cause and diagnose a cache miss

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

1. Change something that alters the prefix. The most reliable option is to **change the model or the reasoning effort** in the model picker (as noted in the prompt caching notes, even switching reasoning level behaves like switching models and breaks the cache). Then send another short follow-up:

   ```
   Summarize the CommentList component in one sentence.
   ```

2. Back in the **Cache Explorer**, select this newest turn. Notice the cache-hit percentage and reused tokens drop sharply compared with the previous turn.

3. Inspect the **Prompt signature**. It summarises the request components (system instructions, tool definitions, messages) and marks the **first divergence** — the point where this request stops matching the previous one.

4. Expand the diverging **Component** to see the text diff and identify exactly what changed. Confirm that the divergence corresponds to the change you made (the model/reasoning configuration), not to your actual message.

5. Try one more cause of your own: enable or disable the `fix` skill from section 2.2, or attach a new file as context, then repeat a follow-up and re-open the Cache Explorer. Observe how a different change moves the first divergence to a different component (tool definitions vs. messages).

> **Important:** Do **not** remove necessary instructions, tools, or context just to raise the cache metric. Response quality and task correctness always come first — the cache is an optimisation, not a goal.

Key Takeaway: A cache miss is not random. The Cache Explorer turns an abstract cost/latency number into a concrete diff, letting you see *which* component changed the prefix. The practical lesson is to keep request context stable within a session — pick your model, reasoning effort, tools, and instructions up front, and start a new session intentionally (`/compact` or a fresh chat) rather than churning configuration mid-conversation.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.